



# RUNNING.CHRISTEL: A Stochastic Hybrid Case-Study Optimizing Battery Pack Usage

Lisa Willemsen<sup>1</sup>(✉) , Anne Remke<sup>1,2</sup> , Boudewijn R. Haverkort<sup>1</sup> ,  
and Johann L. Hurink<sup>1</sup>

<sup>1</sup> University of Twente, Enschede, The Netherlands

{l.c.willemsen,b.r.h.m.haverkort,j.l.hurink}@utwente.nl

<sup>2</sup> University of Münster, Münster, Germany

anne.remke@uni-muenster.de

**Abstract.** Over the last two decades, batteries have become essential components in many high-tech systems, enabling the storage of (electrical) energy for later use. However, the high costs and scarcity of the materials used for high-end batteries make their efficient use and dimensioning a crucial aspect in system design. Enabling such dimensioning requires accurate modeling of the battery behavior under realistic workload conditions. In this paper we use the Kinetic Battery Model (KiBaM) to effectively capture key behavioral aspects of batteries at reasonable modeling costs. Also an accurate battery workload model is needed that describes the demand of energy over time; such workload models often include stochastic elements. Timed automata models have been used to evaluate battery lifetimes under (primarily) *deterministic* workloads for only small battery configurations. The previously proposed approach required a discretization that led to a computational error that could not be quantified in general. Instead, this paper adopts a stochastic hybrid modeling (SHM) approach to better capture battery dynamics as well as *stochastic* workloads. This paper, hence, presents an exploration of state-of-the-art SHM methods and tools for the analysis of battery systems under stochastic workloads, and an investigation of the advantages and disadvantages of these techniques.

**Keywords:** Batteries · Stochastic Hybrid Models · Optimization

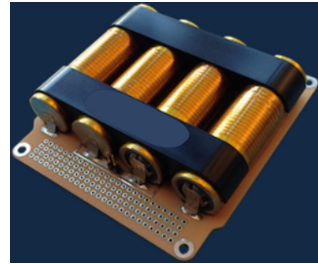
## 1 Introduction

Batteries play an important role in a wide range of wireless high-tech systems and applications. They also enable significant sustainability contributions, by their ability to temporarily store energy, hence their capability to decouple energy generation from energy use. In modern households, for example, solar panels combined with stationary batteries allow for the storage and later use of self-generated solar electricity, particularly during periods when sunlight is not

available. In addition, batteries also support grid stability by enabling energy balancing, which improves the overall safety and reliability of the electricity grid [26]. Despite these advantages, the relatively high cost and limited availability of battery materials necessitate a careful and efficient use of batteries to maximize their utility. Accurate modeling of battery behavior is crucial for optimizing performance in various systems, such as smart home networks and space exploration technologies. In particular, understanding battery aging and optimizing their lifetime are essential to ensure that tasks can be completed reliably and timely within the constraints of a given battery configuration. A battery model that accounts for key non-linear battery-internal phenomena, such as the rate-capacity effect and the recovery effect (to be described below) is therefore essential. The so-called Kinetic Battery Model (KiBaM) [35] has proven to be effective in representing such non-linear behavior, making it a valuable tool in battery system modeling.

In addition to the battery model itself, an effective and realistic workload model is required to capture the stochastic nature of energy demand and use. Previous studies, cf. [27, 28], have employed a deterministic approach to model the combination of two batteries and evaluate their lifetimes under various fixed workload patterns using model checking techniques for timed automata, thereby using the tool UPPAAL. This approach relies on a discretization of the battery storage levels and can predict a system's lifetime for deterministic workloads. However, it is less suitable for stochastic workloads due to the inherent variability. Furthermore, the scalability of this approach is limited by state-space explosion, constraining evaluations to just two batteries. To address these limitations, this paper proposes a stochastic hybrid modeling (SHM) approach, aiming to capture the complex interactions between the battery systems and less predictable stochastic workloads. We explore different simulation and analysis tools capable of extracting meaningful insights from SHM models, with the goal of improving battery system design and management under real-world conditions.

This paper is dedicated to the 60th birthday of Christel Baier. Its topic is a deliberate choice, as it connects us to Christel Baier. Since the beginning of our cooperation, we have been working on stochastic models, bringing together expertise from the formal methods field within theoretical computer science, and from operations research, including stochastic processes, Markov chains, Markov decision processes and computer performance and reliability. We are thankful to Christel for her open attitude and admire her willingness to really get to the bottom of interesting research questions. Also, we appreciate the many things we learned while cooperating, as well as the many fruitful and pleasant days of cooperation, in Bonn, Dresden, Aachen and Twente, as well as during other



**Fig. 1.** 4-cell battery pack.

events around the globe. Anne is still grateful for her time as visiting scholar in Dresden with Christel's group in 2014.

*Outline.* In Sect. 2 we discuss related work, before we address battery models in Sect. 3. In Sect. 4 we briefly present stochastic hybrid models and in Sect. 5 we discuss tools that support their analysis. Some of the considered tools support reinforcement learning, which we touch upon in Sect. 6. In Sect. 7 we introduce a case-study addressing the lifetime of a system with  $N$  batteries. The implementation of the case-study is described in Sect. 8 and results are discussed in Sect. 9. Finally, we conclude the paper in Sect. 10.

## 2 Related Work

Over the past decades many lines of research focused on quantifying safety properties for safety-critical systems. To provide a quantitative analysis, multiple formalisms for stochastic hybrid models have been proposed, which extend hybrid automata [4, 23] with stochastic components. For example, [7] introduces the class of stochastic timed automata, which resolves the inherent non-determinism in timed automata stochastically. A more general definition of stochastic hybrid models named CTSHA is given in [34], where continuous variables evolve according to a stochastic differential equation and two stochastic kernels are used to determine the jump times and destinations within the execution of the hybrid automaton. Time-discrete versions of CTSHA are discussed in [1]. Further, [14, 46] extend subclasses of HA with random clocks, which model a stochastic resolution of continuous non-determinism. While the above formalisms allow to accurately model the interplay of discrete, continuous and stochastic behavior, the formal analysis of such models is a challenging task due to their inherent complexity. This issue has been tackled via other lines of work which focus on the simulation of stochastic hybrid models. For instance, in [39, 40] hybrid Petri net models of systems are simulated to train a reinforcement learning agent to take optimal decisions in the given system.

The tool UPPAAL [6] is widely used and offers a range of tools and verification techniques for networks of timed automata. Stochastic extensions for more expressive models can be evaluated with *statistical model checking* (SMC) and optimal strategies can be identified and constructed [5, 13]. Other tools for the analytical evaluation of stochastic hybrid models focus on *discrete-time* stochastic systems [11, 47]. Algorithms for non-linear continuous dynamics are provided in [45]. The tools applied in this paper are introduced in more depth in Sect. 5.

Next to these research-focused tools, the simulation tool *Simulink* is well-established in both industry and research [36]. The combination of deductive verification and SMC-based learning have been considered in [2, 3] to obtain safe and resilient systems in *Simulink*. A formal semantics is provided for a subclass of *Simulink* models using a transformation in [9].

The lifetime of a system with two attached batteries is analyzed in [27, 28], which corresponds to the time in which at least one battery is not empty. It is assumed that jobs may have different service times and energy demands. During

the system execution, jobs are served sequentially, where idle periods may occur. To determine a schedule maximizing the system's lifetime, [27, 28] model the two batteries via a discretized version of the KiBaM and implement the system as a priced-timed automata (PTA). This PTA is then analyzed using UPPAAL Cora resulting in a control schedule which determines which battery should be used to serve the job in order to achieve a maximal overall system lifetime. If a battery runs empty while serving, it is marked empty and the other battery, if still non-empty, continues the job. Note that a battery that has been marked empty cannot be chosen again. Furthermore, the work presented in [27, 28] compares the schedule maximizing the lifetime of the battery against other scheduling techniques such as round robin or sequential discharging. To avoid state-space explosion in the model-checking process, the number of scheduling decisions has to be kept low in [27, 28]. Thus, the maximum battery capacity and the number of batteries needs to be restricted. Furthermore, the discretization of the KiBaM leads to small deviations to the behavior of the original KiBaM. The above case-study is extended in [8] by considering a stochastic extension of the KiBaM [25]. This extension is applied to a nano-satellite of type GOMX-3, which is also considered in [56]. Here, the satellite is considered to have one battery and is modeled by a stochastic timed automaton in UPPAAL Stratego. The work then uses reinforcement learning to obtain strategies maximizing the longevity of a battery.

### 3 Battery Modeling

Formally speaking, a battery consists of multiple electrochemical cells, however, a single cell is also often called a battery. Within a cell, an electrochemical reaction converts the stored chemical energy into electrical energy. For a more detailed overview, we refer to [29]. Depending on the chemical reaction type, some batteries can be recharged, while others cannot. In general, the *maximal capacity*  $C_{max}$  of a battery indicates how much energy can be stored in the battery. Furthermore, the *effective capacity* states how much energy can still be used during the current discharge-cycle. Due to the electrochemical processes, modeling the behavior of batteries is complex. In particular, during discharge two main non-linear effects occur [30]:

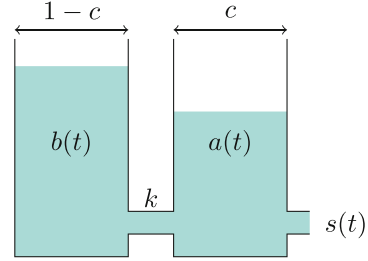
- If a battery (cell) is discharged with a high current, the voltage drops slowly and the resulting effective capacity of the battery is lower, which is called the *rate-capacity effect*.
- The effective capacity can recover in an idle period after periods of discharge. This *recovery effect* may again increase the perceived battery lifetime.

Clearly, a linear battery model is unable to capture these effects but instead models only an ideal case, where voltage remains constant during discharge, and the effective capacity drops to zero as soon as the battery is empty (and not earlier than that). To better capture the above two physical effects, a large variety of models have been proposed ranging from simple analytical models, e.g.,

based on Peukert's law [42], to detailed electrochemical models, as summarized in [12, 30]. Detailed electrochemical models allow for more precise modeling but are also more complex and more difficult to use in practice [32].

In the remainder of this section we discuss an intuitive analytical model, the so-called *Kinetic Battery Model* (KiBaM) [35], which offers a good balance between accuracy and complexity. The KiBaM, as depicted in Fig. 2, assumes that the charge of the battery is distributed over two wells, called available ( $a$ ) and bounded ( $b$ ) charge.

The proportional width of the available charge is given by a parameter  $c \in (0, 1)$  and the proportional width of the bounded charge well is  $1 - c$ . When applying a load  $s$  to the battery, the energy is taken (consumed) from the available charge. Meanwhile, the available charge well is refilled from the bounded charge, via a pipe with fixed conductance  $k$ . The change of charge within each well is described by differential equations with parameters  $c$ ,  $k$ , and  $C_{max}$ , as follows:



**Fig. 2.** Graphical representation of the KiBaM.

$$\begin{aligned}\dot{a}(t) &= k \cdot \left( \frac{b(t)}{1-c} - \frac{a(t)}{c} \right) + s(t), \\ \dot{b}(t) &= k \cdot \left( \frac{a(t)}{c} - \frac{b(t)}{1-c} \right),\end{aligned}\tag{1}$$

where  $a(t)$  and  $b(t)$  denote the available and bounded charge at time  $t$  and  $s(t)$  is the load that is applied to the battery at time  $t$ . If the available charge is at its upper (lower) boundary and  $s(t)$  is larger (smaller) than  $-k \cdot \left( \frac{b(t)}{1-c} - \frac{a(t)}{c} \right)$ ,  $\dot{a}(t)$  is defined to be 0.

In the following we assume that the initial charge is distributed over the wells according to the proportional width, i.e.,  $a(0) = c \cdot C_{max}$  and  $b(0) = (1-c) \cdot C_{max}$  and we assume that the workload  $s(t)$  only changes at given discrete time points  $t_1, t_2, \dots, t_n$ . Note, that the KiBaM model indeed captures certain non-linear behavior, like the *rate-capacity effect* and the *recovery effect*.

## 4 Stochastic Hybrid Automata

Hybrid automata (HA) [24] are a well-established formalism to model complex real-world applications, as they can capture the interplay of discrete and continuous behavior. Naturally, HA include inherent non-determinism via multiple possible discrete or continuous actions and can be used to perform a qualitative analysis, however, in many cases a quantitative analysis is desired. We give a formal definition below.

**Definition 1. (Hybrid Automata: Syntax [4]).** A HA  $\mathcal{H}$  is a tuple  $(Loc, Var, Flow, Inv, Edge, Init)$ , with the following components:

- *Loc* denotes a finite set of locations.
- *Var* is a finite set of real-valued variables. A function  $v : Var \rightarrow \mathbb{R}$  assigns a real valuation to each variable. The set of all valuations is denoted  $V$ . The state of a HA is given by the tuple  $(q, v)$ , which consists of a location  $q \in Loc$  and a valuation  $v \in V$ . The location of the current state is called the control location.
- *Flow* is a labeling function that assigns a set of flows to each location. Each flow  $f \in Flow(q)$  is given by a function  $f : \mathbb{R}_{\geq 0} \rightarrow V$  and specifies the evolution of the continuous state. It is required that flows are time-invariant. Thus, for all locations  $q \in Loc$ , all flows  $f \in Flow(q)$  and  $t \in \mathbb{R}_{\geq 0}$  also  $(f + t) \in Flow(q)$ , with  $(f + t)(t') = f(t + t')$ , for all  $t' \in \mathbb{R}_{\geq 0}$ .
- *Inv* is a labeling function which assigns an invariant  $Inv(q) \subseteq V$  to each location  $q \in Loc$ . The invariant indicates the domain of the continuous state within the current location.
- *Edge*  $\subseteq Loc \times 2^V \times (V \rightarrow V) \times Loc$  is a finite set of discrete transitions or jumps. For a jump  $(\ell, g, r, \ell') \in Edge$ ,  $\ell$  and  $\ell'$  are its source resp. target locations,  $g$  its guard, and  $r$  its reset.
- *Init* is a non-empty set of initial states.

Note that compared to [4], we omit labels from the above definition. Adding labels would allow us to specify the parallel compositions of HA which, however, is not required for the course of this paper.

The execution of an HA distinguishes between discrete and timed steps, where discrete steps correspond to taking a transition and timed steps correspond to the passage of time within the control location. A discrete step can only be taken if the continuous state fulfills the specified guard and invariant at the target location. Similarly, a timed step is only possible if a passage of time would not violate the invariant of the current location.

**Definition 2. (Hybrid Automata: Semantics [4]).** For a HA  $(Loc, Var, Flow, Inv, Edge, Init)$ , the semantics of a discrete step is:

$$\frac{(q, \mu, q') \in Edge \quad (v, v') \in \mu \quad v \in Inv(q) \quad v' \in Inv(q')}{(q, v) \rightarrow (q', v')} Rule_{discrete}.$$

The timed step semantics of the automaton is given by:

$$\frac{f \in Flow(q) \quad f(0) = v \quad f(t) = v' \quad \forall 0 \leq t' \leq t : f(t') \in Inv(q)}{(q, v) \xrightarrow{t} (q, v')} Rule_{time}.$$

A path  $\varrho_{\mathcal{H}}$  of an HA  $\mathcal{H}$  is a finite or infinite sequence of states, where changes in the state happen according to either a timed step or a discrete step.

HA have been extended with stochastic components [1, 7, 15, 21, 34] which (partially) resolve the HA's inherent non-determinism. Several approaches to include stochastic behavior have been generalized as well as classified w.r.t. their

expressivity in [53, 54]. In particular, they extend HA with stochastic kernels in various ways to allow for stochastic decisions on the discrete or continuous action, as summarized in the following definition.

**Definition 3. (Stochastic Hybrid Automaton: Syntax ).** *A stochastic hybrid automaton is a tuple  $\mathcal{S} = (\mathcal{H}, \Psi)$ , where:*

- $\mathcal{H} = (Loc, Var, Flow, Inv, Edge, Init)$  is a hybrid automaton,
- $\Psi$  is a set of stochastic kernels [31].

Depending on how the stochastic kernels are used in the semantics, five different subclasses of stochastic hybrid automata can be identified. To keep this paper focused, we refer the interested reader to [53, 54] and do not further discuss these differences here.

## 5 Tools for SHA

In the following we present the tools for the analysis of SHA that are used to evaluate the case-study in this paper.

**MODEST TOOLSET.** The MODEST TOOLSET [21, 22] provides a *single-formalism, multiple solution* approach. It contains a collection of tools suitable for the analysis of different subclasses of networks of stochastic hybrid automata (SHAs) [21]. In general, SHAs allow to use continuous time and to specify the evolution of variables according to differential (in)equations. Moreover, choices within the execution of the model can be resolved non-deterministically or probabilistically. While the MODEST TOOLSET supports various input formats, models are typically specified using the MODEST language, which is inspired by process algebras. The MODEST TOOLSET includes the **modes** [10] statistical model checker which supports SHM with linear dynamics as well as lightweight scheduler sampling [33] for non-hybrid models [17, 18]. Further, the MODEST TOOLSET includes the tool **prohver** [21], which relies on a modification of PHAVer [19] and computes safe upper bounds for maximum reachability properties.

**RealySt.** The tool **RealySt**<sup>1</sup> [14, 15] is capable of computing maximal reachability probabilities for the class of rectangular hybrid automata extended with random clocks. It uses a combination of forward reachability analysis with backward refinement to optimize the reachability probabilities. For the flowpipe-based reachability analysis **RealySt** uses the library **HyPro** [44]. Moreover, the GNU Scientific Library (GSL) [20] provides Monte Carlo integration methods, which are used during the analysis performed in **RealySt**.

**Simulink Toolbox.** **Simulink** [36] is an industrially well established, graphical modeling language for hybrid systems. **Simulink** models consist of blocks with discrete or continuous behavior that are connected by so-called signals. The **Simulink** block library provides a large set of predefined blocks, from arithmetic over control flow blocks to integrators and complex transformations. The

<sup>1</sup> Available at <https://zivgitlab.uni-muenster.de/ag-sks/tools/realyst>.



reinforcement learning Toolbox [50] provides an RL agent block, which enables executing RL algorithms within Simulink. It periodically samples observations, provides rewards and chooses actions.

## 6 Reinforcement Learning

As some of the above-mentioned tools apply Reinforcement Learning (RL), we give a brief overview on the topic. RL is a class of machine learning methods in which an agent learns via trial and error to take actions in a dynamic environment in order to maximize some given reward [49]. Often the environment is expressed as a Markov decision process (MDP). An MDP [43] is defined by a tuple  $(S, A, R, P)$ , where  $S$  is a set of states,  $A$  is a set of actions,  $R \subset \mathbb{R}$  is a set of rewards, where  $R_a(s, s')$  indicates the reward for a transition from state  $s$  to state  $s'$  according to action  $a$ . Finally,  $P$  specifies the transition probabilities of the MDP and  $P_a(s, s')$  gives the probability of moving from state  $s$  to  $s'$  in case action  $a$  has been chosen. The agent interacts with the environment at discrete time steps. Each step  $t$ , the agent observes the current state  $s_t \in S$  of the environment and chooses an action  $a_t \in A$ , which may depend on the current state  $s_t$ . The environment then responds with the state  $s_{t+1} \in S$  that was reached as well as a numerical reward  $R_{a_t}(s_t, s_{t+1}) \in R$  which is associated with the transition from state  $s_t$  to state  $s_{t+1}$  via the action  $a_t$ . A so-called *policy*  $\pi(a|s)$  of an RL agent indicates the action  $a$  that is taken when observing state  $s$ . The aim of RL is that the agent learns a policy  $\pi(a|s)$ , which maximizes the cumulative reward.

Two well-established algorithms for RL are *Q-Learning* [51] and *Deep Q-Learning* (DQN) [38]. The first maintains a discrete *Q-Table*, where each state-action pair is explicitly stored. Thus, the  $(i, j)$ -th entry of the table indicates the reward for action  $j$ , when being in state  $s_i$ . During the training, the entries of the Q-Table are updated according to the observed rewards and actions. After training, in state  $s_i$ , the action  $a$  is chosen for which the Q-Table indicates the highest reward. Q-Learning requires discrete and finite state-spaces, as well as discrete and finite action spaces. DQN replaces that Q-Table by a deep neuronal network. This allows to handle high-dimensional and continuous state-spaces.

## 7 Case-Study: An Optimal Route for RUNNING.CHRISTEL

As a case-study, we address a new research project of the Algebraic and Logic Foundations of Computer Science group at the TU Dresden, which aims to find the most efficient route to run around the world in an interactive way. To achieve this, a stochastic model of a runner is developed, denoted RUNNING.CHRISTEL. Next to that, a nano-satellite is being employed, as previously modeled by her friends and long-time co-authors Hermanns and Haverkort [41, 48].

The nano-satellite is equipped with multiple batteries and an integrated camera. It orbits the earth and carries out three tasks: (1) its camera takes a batch of pictures of the earth's surface, (2) the images of the batch are compressed



to reduce their overall size, and (3) the compressed images are sent to a base station in Dresden, where they are used to find the optimal route for RUNNING.CHRISTEL. After the compressed images have been sent, the procedure repeats as long as at least one battery in the satellite has enough charge to operate the system.

To take as many images as possible, the system's lifetime should be maximized, i.e., the time in which at least one of the on-system batteries has some charge left. This can be achieved by switching between the batteries to make use of the non-linear recovery effect of batteries. The case-study at hand uses the approach of [27, 28] and aims at predicting the best switching strategy (between the batteries) such that the overall lifetime of the nano-satellite reaches at least a certain threshold  $L$ .

### 7.1 Modeling RUNNING.CHRISTEL

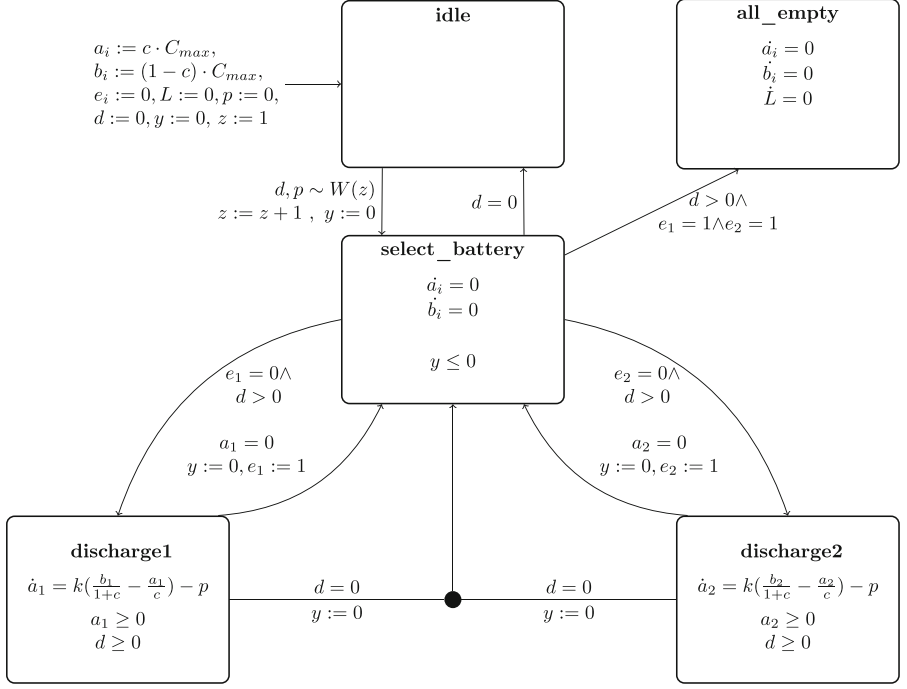
Within the satellite a controller decides which of the batteries is used as power source for an incoming job, based on a given workload model that characterizes the arrival time of jobs, their power demand and required service time. We formalize the workload model as follows.

**Definition 4. (Workload Model).** *A workload model  $W$  for horizon  $h$  is a sequence of tuples*

$$W = [(I_i, arr_i, dur_i, dem_i)]_{1 \leq i \leq h},$$

*where the  $i$ -th tuple represents a job  $i$  with job ID  $I_i$ , arrival time  $arr_i$ , required service time  $dur_i$  and power demand  $dem_i$ .*

In the following, we denote the workload specification of a job  $k$  for a given workload model  $W$  as  $W(k)$ . While serving the jobs according to the given workload model  $W$ , the controller can only choose a non-empty battery. If a battery runs empty while serving, it is marked as empty and a non-empty battery is chosen immediately to continue the job. The lifetime of the system corresponds to the time in which at least one battery is still not marked as empty. We aim to identify the lower bound of the system's maximal lifetime, hence we only consider one charging cycle of the batteries and assume that the batteries are not recharged. Depending on the decisions of the controller, different lifetimes can be reached. To evaluate which lifetimes are possible for the workload model  $W$  representing jobs 1 to  $h$ , the system can be modeled as an hybrid automaton  $\mathcal{H}_W$ . An example for an HA modeling a 2-battery system can be found in Fig. 3. Table 1 indicates the initial values for each variable as well as their flow that holds unless given otherwise in Fig. 3. Here, batteries are modeled by the KiBaM which requires three continuous variables for each battery:  $a_i$  and  $b_i$  to keep track of the available resp. bounded charge of the  $i$ -th battery and  $e_i$  to track whether the  $i$ -th battery is marked as empty (in which case we set  $e_i = 1$ ; initially, all  $e_i = 0$ ). A battery is marked as empty as soon as its available charge runs empty for the first time. Additionally, five continuous variables are needed to monitor the state of the system: the stopwatch  $L$  tracks the lifetime of the system and is



**Fig. 3.** HA model for a system with 2 batteries. If not stated otherwise in the location, variables evolve according to Table 1.

**Table 1.** Overview of continuous variables, their derivatives and initial conditions.

| Variable | Flow                                              | Initial condition       |
|----------|---------------------------------------------------|-------------------------|
| $a_i$    | $\dot{a}_i = k(\frac{b_i}{1+c} - \frac{a_i}{c})$  | $c \cdot C_{max}$       |
| $b_i$    | $\dot{b}_i = -k(\frac{b_i}{1+c} - \frac{a_i}{c})$ | $(1 - c) \cdot C_{max}$ |
| $e_i$    | $\dot{e}_i = 0$                                   | 0                       |
| $d$      | $\dot{d} = -1$                                    | 0                       |
| $p$      | $\dot{p} = 0$                                     | 0                       |
| $L$      | $\dot{L} = 1$                                     | 0                       |
| $y$      | $\dot{y} = 1$                                     | 0                       |
| $z$      | $\dot{z} = 0$                                     | 1                       |

paused as soon as all batteries are marked as empty. The variable  $z$  stores the id of the job that is currently served,  $p$  indicates the power demand of the job that is currently served and  $d$  the remaining serving time of that job. Finally,  $y$  is an auxiliary variable.

If we have  $N$  batteries there are  $3N + 5$  continuous variables in the hybrid automaton. The HA has  $N + 3$  locations and initially, the control location of the

HA is *idle*, where it remains until the arrival of  $W(1)$ . Afterwards the system moves from location *idle* to location *select\_battery* and the values for  $p$  and  $d$  are set to the specifications of  $W(1)$ . As long as there exists an  $i \in \{1, \dots, N\}$  with  $e_i = 0$ , the systems moves immediately from *select\_battery* to a location *discharge\_j*, where  $e_j = 0$  with  $j \in \{1, \dots, N\}$  holds. This corresponds to choosing battery  $j$  to serve the current job. Location *discharge\_j* models the discharging of the chosen battery. From location *discharge\_j* we move to location *select\_battery* as soon as either  $d = 0$  holds, which corresponds to successfully completing the job, or as soon as  $a_j = 0$ . In the latter case, the variable  $e_j$  is set to 1, meaning that battery  $j$  can no longer be chosen for future jobs. If  $d = 0$  location *select\_battery* is left immediately to location *idle*. If  $d > 0$  we either move immediately to location *all\_empty* if for all  $i \in \{1, \dots, N\}$  we have  $e_i = 1$ , or to location *discharge\_k* with  $e_k = 0$  otherwise.

After moving to location *idle*, the process repeats with the next job according to the workload profile. Note that the horizon of the applied workload model is chosen in such a way that regardless of the scheduler which resolves the inherent non-determinism between choosing a discharge location, location *all\_empty* is eventually reached.

## 7.2 Workload Model

In the following we parametrize the workload model. Note that the chosen values are primarily chosen to highlight the stochastic impact and do not necessarily reflect reality. During its lifetime, the satellite orbits the earth and takes images of its surface for exactly two hours and has a power demand of 0.2 W. After two hours of taking images, the satellite starts compressing the images and moves to a position above Dresden from where it can send the images back to earth.

The size of the compression and with that the time until the full batch is sent to earth is not fixed, but strongly depends on the content of the images. Thus, the time taken for compression and sending may vary, where we assume it to be uniformly distributed between 1 and 2 h. Furthermore, the power demand for compressing and sending is 0.9 W.

After the images have been sent, the satellite immediately continues with the task of taking images. Thus, our workload model of horizon  $h$  is given by

$$W_s = [(1, 0, dur_1, dur_1, dem_1), (2, dur_1, dur_2, dem_2), \\ \dots, (k, arr_{k-1} + dur_{k-1}, dur_k, dem_k), \\ \dots, (n, arr_{h-1} + dur_{h-1}, dur_h, dem_h)], \text{ where}$$

$$dem_i = \begin{cases} 0.2W, & \text{if } i \bmod 2 = 0, \\ 0.9W, & \text{otherwise.} \end{cases} \text{ and } dur_i = \begin{cases} 2h, & \text{if } i \bmod 2 = 0, \\ \text{Unif}(1, 2)h, & \text{otherwise.} \end{cases}$$

We also consider a deterministic workload model taken from [27, 28], where all jobs have a required service time of 1h, a power demand of 0.25W and no idling time occurs between jobs. Thus, for an  $h \in \mathbb{N}$ :

$$W_d = [(1, 0, 1, 0.25W), (2, 1, 0.25W), \dots, (h, h, 1, 0.25W)].$$

Since  $W_d$  is purely deterministic,  $\mathcal{H}_{W_d}$  is a hybrid automaton as described in Sect. 4. In contrast, the workload model  $\mathcal{H}_{W_s}$  includes stochasticity and  $\mathcal{H}_{W_s}$  is a SHA. If  $W_s(z)$  corresponds to the job which compresses images and sends them to earth, the required service time of the sending and compression process is set according to a uniform distribution.

The maximal lifetime of the system can possibly be improved by choosing only batteries with sufficient effective capacity for the current job. However, precomputing the power demand requires solving the differential equations of the KiBaM. In general, this is only possible for a deterministic workload model, but not if the required service time is sampled from a probability distribution.

*Numerical Parameters.* We assume that the nano-satellite is equipped with a battery system similar to Fig. 1 and has four attached batteries, where each has a nominal voltage of 3.6 V and range in capacity from 1800 mAh to 3600 mAh. As this corresponds to a range from 6.48 Wh to 12.96 Wh, we fixed the capacities of the batteries in this case-study such that the first battery ( $B_1$ ) has a capacity of 7 Wh, the second battery ( $B_2$ ) has a capacity of 8.5 Wh, the third one ( $B_3$ ) a capacity of 10 Wh and the last battery ( $B_4$ ) a capacity of 12 Wh.

Since each battery is modeled by the KiBaM, the systems lifetime strongly depends on the choice of the parameters  $c$  and  $k$ , as they govern the recovery effect of each battery (see Sect. 3). It follows directly from Equation (1) that the flow from the bounded charge to the available charge is maximal if the height difference between the two wells is maximal, i.e., if  $a(t) = 0$  and  $b(t) = (1 - c) \cdot C_{max}$ . In this case, the flow would be  $k \cdot C_{max}$ . During our evaluation we set  $c = 0.5$  to ensure that enough energy is stored in the available charge to increase the number of batteries from which the scheduler can choose, while also ensuring that the bounded charge stores enough energy, to showcase the recovery effect. According to [27, 28], we set  $k = 0.017$ .

## 8 Modeling the $N$ -Battery System in Different Tools

We implemented the  $N$ -battery system in each of the considered tools given in Sect. 5. The corresponding source files are available online<sup>2</sup>. The Simulink model is explained in Sect. 8.1 followed by a specification of the case-study in the input language of the MODEST TOOLSET in Sect. 8.2 and a discussion of the implementation in RealySt in Sect. 8.3. Even though the case-study of [27, 28] was performed in UPPAAL Cora, for this Festschrift we used academic tools developed at the University of Münster and the University of Twente.

<sup>2</sup> <https://zivgitlab.uni-muenster.de/ag-sks/tools/misc/execution-scripts/2025-04-optimal-battery-pack-usage>.

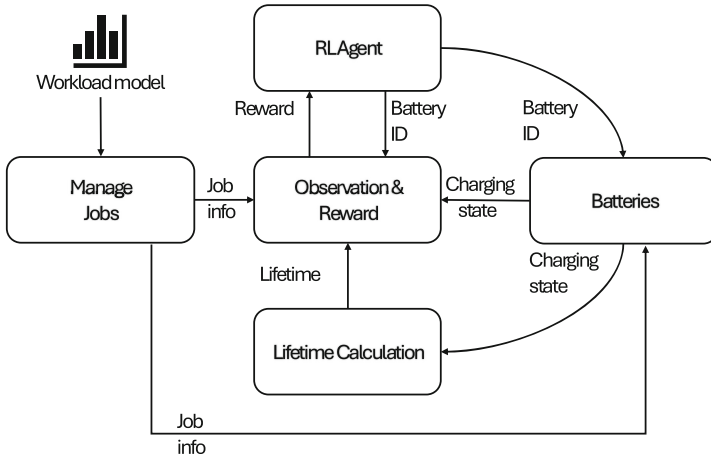
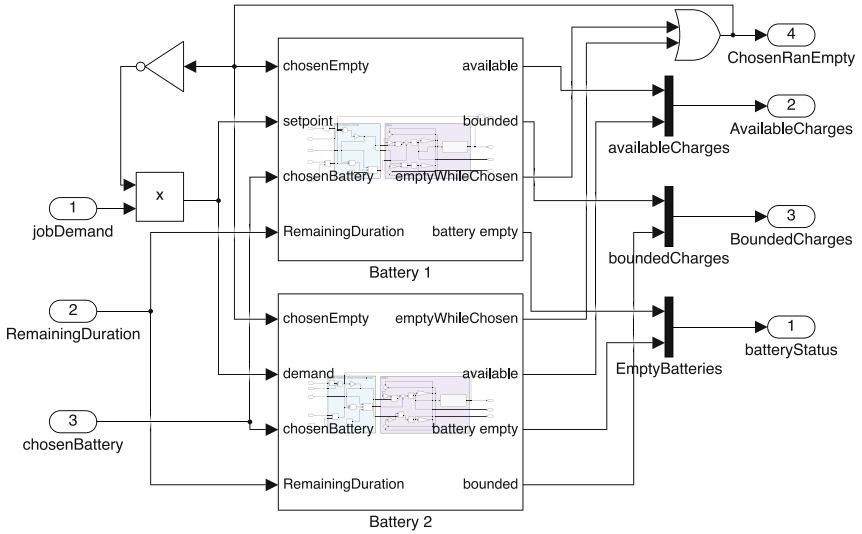
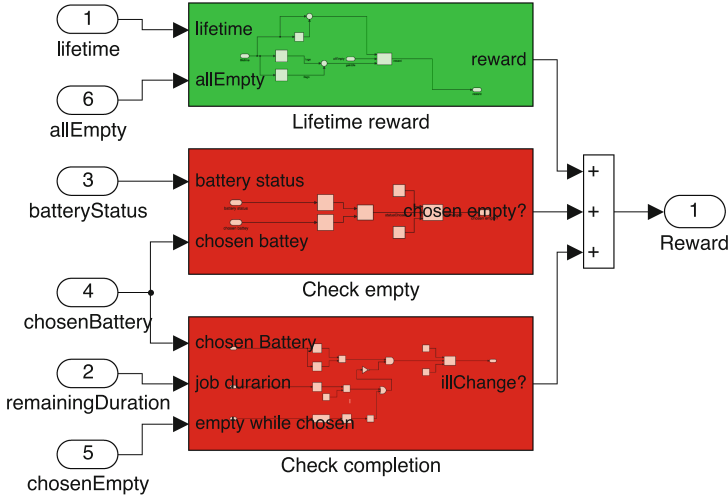


Fig. 4. Schematic overview on the setup of the Simulink Model.

### 8.1 Setup Simulink

The Simulink model shown in Fig. 4 consists of five subsystems: **ManageJobs**, **ObservationAndReward**, **Batteries**, **RL agent** and **LifetimeCalculation**. Note that this model can easily be adapted for a different number of batteries and for different workload models. In the following we describe the tasks assigned to the different subsystems. **ManageJobs** processes the given workload model (either deterministic or stochastic) and forwards information on the current job  $W(z)$ , i.e., its demand and its remaining service time, to **ObservationAndReward** and **Batteries**. **ObservationAndReward** collects and processes information from all other subsystems and provides the system state and a reward to the **RL agent**.

Since Simulink performs discrete-time simulation, the **RL agent** makes a decision after each time-step. This decision is based on its observations (as explained below) and identifies a battery to be used until the next decision. Note that in case a battery runs empty or the job is completed in-between two time steps, the model evolution is paused, hence, the lifetime and the available and bounded charge of the batteries do not evolve further until the next decision of the agent. The decision of the **RL agent** is forwarded to **ObservationAndReward** and to **Batteries**. The latter uses the input from the **RL agent** and the received job information, to initiate the discharging of the corresponding battery. Recall from Sect. 7 that each battery is modeled as a KiBaM in its own subsystem (see Fig. 5b). Finally, **LifetimeCalculation** computes the current lifetime of the system and performs the statistical evaluation. Thus, in case the stochastic workload model is applied, subsystem **Lifetime Calculation** checks whether the lifetime surpasses a given threshold. For the stochastic workload model, we compute the Wilson-Score Confidence Interval [55].



**Fig. 5.** Two subsystems in a Simulink model of a 2-battery system.

We run **Simulink** with a variable-step solver with a maximum step size of 0.025. Differential equations are solved with the Dormand-Prince method (ode45) [16]. We use the parallel training and simulation option of **Simulink**.

The *RL agent* is implemented as a Deep Q-Network agent and receives a set of observations on the state of the system, which include: the previously chosen

battery, the current number of battery changes, the current lifetime as well as the current job information. Furthermore, the observation includes features based on information on the state of each battery, i.e., the bounded and the available charge, as well as the information whether the battery is marked as empty. The action space of the agent corresponds to the IDs of the  $N$  batteries, ranging from 1 to  $N$ . During training, the features in the observation space are used to learn the weights of the deep neuronal network, which determines the action of the controller. The impact of an observation to the result is driven by a complex interplay of different weights. Thus, it is generally challenging to predict the influence that a single element from the observation space has. Note that the chosen observation space for this paper is not necessarily minimal, but could be reduced without affecting the overall performance of the DQN agent. The importance of single features from the observation space could be evaluated with methods from the field of *explainable AI* (see [37] for a survey). The first layer of the deep neural network used by the agent is an input layer which processes the observations. This is followed by two alternating layers of fully connected units (256 each) and ReLU activation functions. The final layer is a fully connected output layer, which generates a Q-value for each possible action for the given observation. As introduced in [38], Simulink uses a target critic and memory replay. We use a learning rate of  $5 \cdot 10^{-3}$  for the critic and adapt the standard settings of the agent to `ExperienceBufferLength=25 · 10-3`. To enhance learning, we normalize the observations.

The aim is to train the agent, such that its decisions maximize the overall lifetime of the system. For this, a reward function calculates for each action of the agent a reward within a subsystem that is depicted in Fig. 5a and integrated into **Observation & Reward**. The reward is composed out of three different components, where the first is the **Lifetime reward** (indicated in green), which rewards the agent with a value from the interval  $[0, 1]$ , corresponding to the increase in lifetime since the last decision of the agent. Furthermore, we use *reward shaping* to ensure that the agent only takes *valid actions*, i.e., no empty battery is chosen and a new battery is only chosen if either the previous job is completed or the previous battery ran empty. The subsystems **Check empty** and **Check completion** (indicated in red) provide the agent with a high negative reward of  $-2$  in case the last action was not valid.

## 8.2 Setup MODEST TOOLSET

We implemented the HA of Fig. 3 within the Modest Toolset. Hereby, each location of the HA is described as a separate process and the changes between the processes are specified according to the guard conditions. Listing 1.1 shows the specification of location *discharge1* of Fig. 3 in Modest. The invariant of the process specifies the flows for each continuous variable, as well as the conditions on the continuous state. The transitions leading to location *select\_battery* are encoded into the `alt` statement. If a process is changed, Modest allows to reset variables to samples of a given probability distribution. Thus, the specifications



for an HA with  $W_d$  or  $W_s$  in Modest only differ in the specification of the conditions when the process *Idle* is left. Note that for the analysis with **modes**, guard conditions cannot be specified as an open interval. To avoid Zeno behavior, we therefore replace the guard condition  $d > 0$  with  $d \geq 0.000001$ .

**Listing 1.1.** Implementation of location *discharge1* from Fig. 3 in Modest.

---

```

process discharge1()
{
  invariant (der(a1) == (k * (b1 / (1 - c) - a1 / c)) - p
    && der(b1) == k * (a1 / c - b1 / (1 - c))
    && der(a2) == k * (b2 / (1 - c) - a2 / c)
    && der(b2) == k * (a2 / c - b2 / (1 - c))
    && der(p) == 0 && der(d) == (-1) && der(L) == 1
    && der(h) == 1 && der(T) == 1 && a1 >= 0 && d >= 0)
  alt {
    :: when ((d >= eps) && (a1 <= 0))
      tau {= e1 = 1, h = 0 =}; Select_Battery()
    :: when (d <= 0) tau {= h = 0 =}; Select_Battery()
  }
}

```

---

We use the simulation tool **modes** with its Q-Learning extension to analyze whether the lifetime of the system can exceed a given threshold  $L$ . The agent aims at resolving the discrete non-determinism arising in the HA of Fig. 3 such that the probability to exceed the given threshold is maximized. Unless stated otherwise, we use the Q-Learning extension with the following settings:

rate = 0.1, disc-factor = 10, final-rate = 0.05, discount = 1.0,  
epsilon = 0.15, final-epsilon = 0.05.

For the statistical evaluation, again a Wilson-Score Confidence Interval is computed. Note that the agent observes the current sampling of the random variable and the remaining duration of stochastic delays is part of the observation space. Hence, we obtain a prophetic scheduler if a stochastic workload model is applied, which corresponds to the computation of a best case that can be achieved in this setting. For deterministic workload models, we also apply a uniform scheduler to all non-deterministic decisions instead of Q-Learning. In that case each decision is resolved probabilistically, where each choice has the same probability. We then evaluate the maximal threshold  $L$  for which the probability to eventually reach a lifetime of  $L$  is larger than 0. Note that the uniform scheduler returns a probability for reaching a predefined lifetime  $L$ , but is not able to return a schedule.

### 8.3 Setup RealySt

For a proper analytical evaluation of the model in **RealySt**, the support of continuous dynamics in the form of Linear-Time-Invariant (LTI) systems [52] is

required. However, this kind of analysis is not yet supported in **RealySt**, nor in **prohver**. However, by adapting the HA shown in Fig. 3 to continuous variables with constant rates, the tool can be used to determine an over-approximation of the system lifetime. For this, we construct a singular automaton [24] as follows:

The lifetime of the model strongly depends on the KiBaM parameters, as discussed in Sect. 7. To make the model analyzable for the analytical tools, we have to assume a maximal flow between the two wells of the KiBaM and we set  $\dot{a}_i = k \cdot C_{max}$ , when the battery  $i$  is idling and  $\dot{a}_i = k \cdot C_{max} - p$ , otherwise. This over-approximates the lifetime of the model as explained in Sect. 7.2. Moreover, we ensure that  $0 \leq a_i \leq c \cdot C_{max}$  holds at each time. The resulting singular automaton is specified in the input format of **RealySt**. To improve computation time, we replace guards of the form  $d > 0$  by  $d \geq 0.000001$ .

## 9 Evaluation

In this section the lifetime of the nano-satellite is evaluated by checking whether it reaches a certain threshold  $L$ . In Sect. 9.1, we introduce the specifications of our experiments. Section 9.2 considers a two-battery system with a deterministic workload model which is analyzed with analytical tools. In Sect. 9.3 we consider various settings which are analyzed with simulation-based tools.

### 9.1 Specifications

We consider different settings to showcase important aspects in the analysis of our proposed hybrid models with the tools discussed in Sect. 5. We identify the different settings via a tuple of the form  $(N, C_{max}, c)$ , where  $N$  indicates the number of batteries used,  $C_{max}$  the maximal capacity of each battery and  $c$  the chosen value for the parameter  $c$  of the KiBaM. Note that in each setting, we set  $k = 0.017$ , therefore, this parameter is not included in the tuple.

All experiments using **Simulink** or a tool from the MODEST TOOLSET have been conducted on a machine equipped with two AMD EPYC 9354 processors with 32 kernels running at 3.25 GHz and a total of 512 GB of RAM. The experiments using **RealySt** have been conducted on a machine equipped with an Intel<sup>®</sup> Core<sup>™</sup> i7 processor running at  $10 \times 1.7$  GHz with 64 GB of RAM. Unless stated otherwise we train the DQN-agent in **Simulink** for 600 and the Q-Learning agent in **modes** for 10000 episodes, as these two training techniques require a different number of training episodes. In both tools one simulation run is performed with the learned strategy. To simulate **modes** with the uniform scheduler, we use 10000 simulation runs.

**Table 2.** Lifetime obtained for a (2, 5.5 Wh, 0.166) system with deterministic workload and jobs with power demand 0.4 W and service time 1 h. Per setting, 10 simulation runs are performed. The lifetime (LT) is indicated by the maximum and the training time (TT) by the average.

| RealySt | prohver | modes             |                          | Simulink |        |
|---------|---------|-------------------|--------------------------|----------|--------|
|         |         | <i>Q-Learning</i> | <i>Uniform Scheduler</i> |          |        |
| 9.2     | 9.2     | 5.75              | 5.75                     | 5.73     | LT [h] |
| 58.82   | 1.36    | 2.28              | 6.64                     | 94.09    | TT [s] |

## 9.2 Results Obtained by Analytical Tools

As mentioned in Sect. 8.3, both analytical tools **prohver** and **RealySt** do not support the analysis of systems with linear dynamics such as the KiBaM (yet). However, by over-approximating the recovery effect modeled in the KiBaM via a singular automaton, an upper bound for the lifetime can be determined. Table 2 compares the results achieved by the different tools for the setting (2, 5.5 Wh, 0.166) when applying a deterministic workload model, where jobs with demand 0.4 W and a required service time of 1h arrive consecutively. While the analytical models over-approximate the recovery effect as explained above, **Modes** and **Simulink** analyze the lifetime taking into account the non-linear effects modeled by the KiBaM.

The over-approximations provided by **RealySt** and **prohver** both lead to a lifetime of 9.2h. Hereby, the computation time of **prohver** is much smaller than **RealySt**. Also the results obtained by the simulation-based tools coincide, where **modes** is considerably faster than **Simulink**. Due to the deterministic workload model no confidence intervals are provided. The over-approximation results in a lifetime that is approximately 60% larger than the simulated lifetimes. To obtain a meaningful upper bound it could be interesting to investigate a tighter over-approximation of the recovery effect in the KiBaM in future work. Because the over-approximation of the recovery effect increases the lifetime significantly, also the number of scheduling decisions increases drastically. Similar to [27, 28], this leads to a state-space explosion resulting in very high computation times. Hence, in the following, we only consider simulation-based tools which are able to deal with larger state-spaces and with the nonlinear dynamic induced by the KiBaM.

## 9.3 Results Obtained by Simulation

*2- or 3-Battery Systems.* We first consider the setting (2, 5.5 Wh, 0.166) which corresponds to one of the settings in [27, 28]. To highlight the scalability of our model we further extend this setting towards higher values of  $c$  and  $C_{max}$ , as well as towards 3 batteries. Each of these changes leads to a larger amount of scheduling decisions, increasing the complexity of the problem.

**Table 3.** Lifetimes and computation times for settings (i) (2, 5.5 Wh, 0.166), (ii) (2, 5.5 Wh, 0.5), (iii) (2, 11 Wh, 0.5) (iv) (2, 30 Wh, 0.5) and (v) (3, 5.5 Wh, 0.166), for a deterministic workload model  $W_d$ . The computation time (TT) is the average of the 10 training runs performed, and the lifetime (LT) is their maximum.

|       | Simulink | modes      |         |        |
|-------|----------|------------|---------|--------|
|       |          | Q-Learning | Uniform |        |
| (i)   | 11.89    | 11.89      | 11.93   | LT [h] |
|       | 78.36    | 6.83       | 12.57   | TT [s] |
| (ii)  | 30.97    | 30.75      | 30.98   | LT [h] |
|       | 114.87   | 11.42      | 27.87   | TT [s] |
| (iii) | 73.23    | 73.2       | 73.24   | LT [h] |
|       | 202.10   | 24.00      | 59.02   | TT [s] |
| (iv)  | 224.97   | 225.1      | 225.15  | LT [h] |
|       | 602.37   | 64.75      | 155.47  | TT [s] |
| (v)   | 23.58    | 15.8       | 24.05   | LT [h] |
|       | 101.75   | 7.57       | 22.90   | TT [s] |

Table 3 shows results for *setting (i)* (2, 5.5 Wh, 0.166), *(ii)* (2, 5.5 Wh, 0.5), *(iii)* (2, 11 Wh, 0.5), *(iv)* (2, 30 Wh, 0.5) and *(v)* (3, 5.5 Wh, 0.166) in combination with the deterministic workload model  $W_d$ . For each case, we use the uniform scheduler of **modes**, to obtain the maximal possible lifetime for this scenario. For a system with two batteries, both Deep Q-Learning in **Simulink** and Q-Learning in **modes** obtain schedules which maximize the system's lifetime. The computation time of **modes** with Q-Learning outperform **Simulink** in all cases, while the resulting lifetime is similar in all cases, except for *setting (v)*, which is discussed together with two other interesting settings below.

For *setting (i)* an optimal schedule is provided in [27, 28] which is computed for a discretized version of the KiBaM and results in a lifetime of 12.04h. As the source code of [27, 28] is not available, we could not analyze the computation time of UPPAAL on a modern machine. The results of our models are within the 1% error induced by the discretization of the KiBaM for the considered workload model, as indicated by the authors.

For *setting (iv)* the discretization factor which determines the discretization, had to be reduced to 1 in **modes** to allow its Q-Learning agent to learn optimal decisions. This reduces the amount of possible states drastically and implies that it is more challenging to find an optimal solution using Q-Learning for settings with a large state-space. Both, the number of batteries and their maximal capacity increase the state-space, hence, the lifetime obtained via Q-Learning for *setting (v)*, which is a setting with three batteries, is significantly lower than the one obtained via Deep Q-Learning in **Simulink**.

**Table 4.** Probability to reach a lifetime of at least threshold  $L$  for *setting* (i) and a stochastic workload model. Confidence intervals (CI) are Wilson-score with confidence level 0.95 for 5000 simulation runs. Computation time is specified for training (TT) and simulation (ST).

| <b>L</b>        | $\geq 7$     | $\geq 8$     | $\geq 9$    |        |
|-----------------|--------------|--------------|-------------|--------|
| <b>modes</b>    | [1.00,1.00]  | [0.44,0.47]  | [0.00,0.00] | CI     |
|                 | 2.58         | 4.82         | 6.01        | TT [s] |
|                 | 1.01         | 2.09         | 2.88        | ST[s]  |
| <b>Simulink</b> | [1.00, 1.00] | [0.44, 0.47] | [0.00,0.00] | CI     |
|                 | 100.22       | 100.22       | 100.22      | TT[s]  |
|                 | 112.67       | 112.67       | 112.67      | ST [s] |

For each setting, the uniform scheduler in **modes** outperforms the results obtained by Q-Learning. We suppose that this is caused by the high number of possible schedules, where finding the optimal one is very rare and therefore hard to learn for RL agents. Recall that, simulation with a uniform scheduler, do not lead to schedules which maximizes the lifetime. In contrast, applying **modes** with Q-learning or **Simulink** with Deep Q-learning results in a trained agent that provides a scheduling strategy.

When a stochastic workload model, as in Table 4 is applied, the lifetime strongly depends on the samples of the service times. Hence, a maximal lifetime cannot be determined as for the previous case in which a deterministic workload model was applied. Instead, statistical model checking is used to indicate the probability that a certain lifetime is reached. Table 4 shows the results obtained for a stochastic workload model where every second job has a service time that is uniformly distributed over  $(0, 1)$  and a demand of  $0.4W$ . This workload model is applied to the setting  $(2, 5.5 \text{ Wh}, 0.166)$ , where results obtained with **Simulink** and **modes** match. Even though **Simulink** is able to compute results for all three considered lifetimes in one run, i.e., the indicated computation time is only needed once, **modes** again has a lower execution time than **Simulink**.

*4-battery system for RUNNING.CHRISTEL* The simulation tools are able to evaluate a system with four batteries, which was not possible in [28], and is also not possible for the analytical tools considered in this paper. Tables 6 and 5 summarize the simulation results for the 4-battery system with a deterministic and a stochastic workload, respectively.

First, Table 5 shows the lifetimes in a setting with four batteries and a workload model in which the required service time of every second job is fixed to either 1, 1.5 or 2 h. The values in column *Max.* indicate the lifetime that can be achieved if for each battery the available charge and the bounded charge are fully depleted. Similarly, column *Min.* indicates the smallest lifetime achieved by a scheduler that only discharges the available charge of each battery. Note that *Min.* and *Max.* provide a theoretical range for the lifetime, where *Max.* cannot be reached in practice, as it is usually not possible to completely drain a battery.

**Table 5.** Lifetime in hours for a the deterministic workload model  $[(0, 0, 2h, 200W), (1, 2, x_1, 900W), (3, 2 + x, 2h, 200W), (4, 4 + x, xh, 900W), \dots]$  with  $x = \{1, 1.5, 2\}$ , applied to the setting  $(4, [7 \text{ Wh}, 8.5 \text{ Wh}, 10\text{Wh}, 12 \text{ Wh}], 0.5)$ . Trainings have been run 10 times, the time (TT) is the average of these runs and the lifetime the maximum obtained from these runs.

|           | Min.  | Max.  | Simulink | modes      |                   |        |
|-----------|-------|-------|----------|------------|-------------------|--------|
|           |       |       |          | Q-Learning | Uniform Scheduler |        |
| $x = 1$   | 43.26 | 86.53 | 71.08    | 57.6       | 71.2              | LT [h] |
|           |       |       | 222.18   | 17.10      | 56.39             | TT[s]  |
| $x = 1.5$ | 37.5  | 75    | 59.01    | 48.8       | 59.2              | LT [h] |
|           |       |       | 223.51   | 13.12      | 41.20             | TT[s]  |
| $x = 2$   | 34.09 | 68.18 | 51.28    | 43.3       | 52                | LT [h] |
|           |       |       | 216.58 s | 11.29      | 33.89             | TT [s] |

**Table 6.** Probability to reach a life-time of at least  $L$  for the stochastic workload model  $W_s$  in settings  $(4, [7 \text{ Wh}, 8.5 \text{ Wh}, 10 \text{ Wh}, 12 \text{ Wh}], 0.5)$ . Confidence intervals (CI) are computed using the Wilson-score interval method with confidence level 0.95 and 5000 simulation runs. Computation time is specified for training (TT) and simulation (ST).

| L        | $\geq 52$    | $\geq 54$    | $\geq 55$    | $\geq 56$    | $\geq 58$    |        |
|----------|--------------|--------------|--------------|--------------|--------------|--------|
| modes    | [0.53, 0.55] | [0.26, 0.29] | [0.14, 0.16] | [0.07, 0.08] | [0.01, 0.02] | CI     |
|          | 14.23        | 15.54        | 16.21        | 16.64        | 16.97        | TT[s]  |
|          | 7.06         | 7.62         | 7.95         | 8.09         | 8.18         | ST[s]  |
| Simulink | [0.99, 1.00] | [0.97, 0.98] | [0.97, 0.94] | [0.84, 0.84] | [0.46, 0.49] | CI     |
|          | 232.26       | 232.26       | 232.26       | 232.26       | 232.26       | TT [s] |
|          | 130.49       | 130.49       | 130.49       | 130.49       | 130.49       | ST[s]  |

Table 5 indicates that the DQN-agent in **Simulink** is able to learn schedules which lead to lifetimes close to the lifetimes obtained by the uniform scheduler with **modes**. The best results learned by **modes** are achieved for a discretization factor of 1000, which however still leads to significantly lower lifetimes. This can be explained by the large state-space for the system with four batteries.

If the stochastic workload model  $W_s$  is applied, in which every second job has a service time that is uniformly distributed on  $[1, 2]$ , the lifetime strongly depends on the sampled delay. As expected, results for the stochastic workload model lie between the results for  $x = 1$  and  $x = 2$ , i.e., between 52 and 71.2. As higher lifetimes can only be reached, when small service times are sampled, the probability to reach higher lifetime decreases. This can be seen in Table 6, where the confidence intervals for the probability to reach a certain threshold in the lifetime are given. Within **Simulink** it is much more likely to reach a lifetime that exceeds certain threshold than with **modes**. This matches the results of

Table 5, where Q-Learning in `modes` resulted in significantly lower lifetimes for the model with four batteries. This is due to the increasing size of the Q-Table, which makes efficient Q-Learning infeasible. Finally, the results of Table 6 show that the probability that the satellite can be used for at least 56 h is 84%.

## 10 Conclusion

This paper presented how the maximal lifetime of a system equipped with  $N$  batteries can be determined in the presence of stochastic or deterministic workload models. To achieve this, we proposed a hybrid automaton modeling the system and we specified it in four state-of-the-art tools. In particular for this paper, we focused on two simulation-based tools in which a reinforcement learning agent is trained to resolve the decisions on which battery to use. In both tools, the RL agent is trained in such a way that it aims to make decisions leading to a long lifetime of the system. Both tools train with a different RL algorithm. The achieved results are compared to results obtained by two tools providing a formal analysis of hybrid systems.

Our results show that for small systems with a relatively small state-space, both the DQN-agent included in the `Simulink` and the Q-Learning agent trained with `modes`, learn schedules that maximize the system's lifetime even for stochastic workload models. This shows that the discretization of the state-space required for the Q-Learning approach in `modes` has no negative impact on the result for small systems. For such systems `modes` shows a lower runtime and it is guaranteed that only valid actions (i.e. no empty battery is chosen and the used battery is not changed within a job) are carried out. In the `Simulink` model, we use reward shaping which gives a high incentive for the agent to avoid invalid actions. However, it is not guaranteed that they do not occur. Our evaluations also showed that for systems with more than two batteries, the schedule induced by the Q-agent results in significantly lower lifetimes than the schedule induced by the DQN-agent. This shows, that adding more batteries results in a higher dimensional state-space which cannot be efficiently handled by Q-Learning. Adding a pre- or post shield to the RL-agent of the `Simulink` model guarantees that the agent only takes valid actions. In future work we plan to evaluate how a shield can be added and how it influences the learning behavior. Further, we plan to investigate a tighter over-approximation for the KiBaM.

## References

1. Abate, A., Prandini, M., Lygeros, J., Sastry, S.: Probabilistic reachability and safety for controlled discrete time stochastic hybrid systems. *Automatica* **44**(11), 2724–2734 (2008). <https://doi.org/10.1016/j.automatica.2008.03.027>
2. Adelt, J., Bruch, S., Herber, P., Niehage, M., Remke, A.: Shielded learning for resilience and performance based on statistical model checking in Simulink. In: *Proceedings of the first Int. Conf. on Bridging the Gap Between AI and Reality*, pp. 94–118. Springer Nature Switzerland (2024). [https://doi.org/10.1007/978-3-031-46002-9\\_6](https://doi.org/10.1007/978-3-031-46002-9_6)

3. Adelt, J., Herber, P., Niehage, M., Remke, A.: Towards safe and resilient hybrid systems in the presence of learning and uncertainty. In: Proceedings of the 11th Int. Symposium on Leveraging Applications of Formal Methods, Verification and Validation. Verification Principles, pp. 299–319. Springer (2022). [https://doi.org/10.1007/978-3-031-19849-6\\_18](https://doi.org/10.1007/978-3-031-19849-6_18)
4. Alur, R., et al.: The algorithmic analysis of hybrid systems. *Theor. Comput. Sci.* **138**(1), 3–34 (1995). [https://doi.org/10.1016/0304-3975\(94\)00202-T](https://doi.org/10.1016/0304-3975(94)00202-T)
5. Behrmann, G., Cougnard, A., David, A., Fleury, E., Larsen, K.G., Lime, D.: UPPAAL-Tiga: time for playing games! In: Proceedings of the 19th Int. Conf. on Computer Aided Verification, pp. 121–125. Springer (2007). [https://doi.org/10.1007/978-3-540-73368-3\\_14](https://doi.org/10.1007/978-3-540-73368-3_14)
6. Bengtsson, J., Larsen, K., Larsson, F., Pettersson, P., Yi, W.: UPPAAL — a tool suite for automatic verification of real-time systems. In: Alur, R., Henzinger, T.A., Sontag, E.D. (eds.) HS 1995. LNCS, vol. 1066, pp. 232–243. Springer, Heidelberg (1996). <https://doi.org/10.1007/BFb0020949>
7. Bertrand, N., Bouyer, P., Brihaye, T., Menet, Q., Baier, C., Größer, M., Jurdzinski, M.: Stochastic timed automata. *Logical Methods in Computer Science* **10**(4) (2014). [https://doi.org/10.2168/LMCS-10\(4:6\)2014](https://doi.org/10.2168/LMCS-10(4:6)2014)
8. Bisgaard, M., Gerhardt, D., Hermanns, H., Krčál, J., Nies, G., Stenger, M.: Battery-aware scheduling in low orbit: the GomX–3 case. *Formal Aspects Comput.* **31**(2), 261–285 (2018). <https://doi.org/10.1007/s00165-018-0458-2>
9. Blohm, P., Herber, P., Remke, A.: Towards quantitative analysis of simulink models using stochastic hybrid automata. In: Proceedings of the 19th Int. Conf. on Integrated Formal Methods, pp. 172–193. Springer Nature Switzerland (2025). [https://doi.org/10.1007/978-3-031-76554-4\\_10](https://doi.org/10.1007/978-3-031-76554-4_10)
10. Budde, C.E., D’Argenio, P.R., Hartmanns, A., Sedwards, S.: An efficient statistical model checker for nondeterminism and rare events. *Int. J. Softw. Tools Technol. Transf.* **22**(6), 759–780 (2020). <https://doi.org/10.1007/s10009-020-00563-2>
11. Cauchi, N., Abate, A.: Stochy - automated verification and synthesis of stochastic processes: poster abstract. In: Proceedings of the 22nd ACM Int. Conf. on Hybrid Systems: Computation and Control, pp. 258–259. Association for Computing Machinery (2019). <https://doi.org/10.1145/3302504.3313349>
12. Cloth, L., Jongerden, M.R., Haverkort, B.R.: Computing battery lifetime distributions. In: Proceedings of the 37th Annual IEEE/IFIP Int. Conf. on Dependable Systems and Networks, pp. 780–789 (2007). <https://doi.org/10.1109/DSN.2007.26>
13. David, A., Jensen, P.G., Larsen, K.G., Mikučionis, M., Taankvist, J.H.: UPPAAL Stratego. In: Proceedings of the 21st Int. Conf. on Tools and Algorithms for the Construction and Analysis of Systems, pp. 206–211. Springer (2015). [https://doi.org/10.1007/978-3-662-46681-0\\_16](https://doi.org/10.1007/978-3-662-46681-0_16)
14. Delicaris, J., Schupp, S., Ábrahám, E., Remke, A.: Maximizing reachability probabilities in rectangular automata with random clocks. In: Proceedings of the 17th Int. Symposium on Theoretical Aspects of Software Engineering, pp. 164–182. Springer (2023). [https://doi.org/10.1007/978-3-031-35257-7\\_10](https://doi.org/10.1007/978-3-031-35257-7_10)
15. Delicaris, J., Stübbe, J., Schupp, S., Remke, A.: RealySt: A C++ tool for optimizing reachability probabilities in stochastic hybrid systems. In: Proceedings of the 16th EAI Int. Conf. on Performance Evaluation Methodologies and Tools, pp. 170–182. Springer Nature Switzerland (2024). [https://doi.org/10.1007/978-3-031-48885-6\\_11](https://doi.org/10.1007/978-3-031-48885-6_11)
16. Dormand, J.R.: Numerical Methods for Differential Equations: A Computational Approach. CRC Press, 1st edn. (1996). <https://doi.org/10.1201/9781351075107>



17. D'Argenio, P., Legay, A., Sedwards, S., Traonouez, L.-M.: Smart sampling for lightweight verification of Markov decision processes. *Int. J. Softw. Tools Technol. Transfer* **17**(4), 469–484 (2015). <https://doi.org/10.1007/s10009-015-0383-0>
18. D'Argenio, P.R., Hartmanns, A., Sedwards, S.: Lightweight statistical model checking in nondeterministic continuous time. In: Margaria, T., Steffen, B. (eds.) *ISoLA 2018*. LNCS, vol. 11245, pp. 336–353. Springer, Cham (2018). [https://doi.org/10.1007/978-3-030-03421-4\\_22](https://doi.org/10.1007/978-3-030-03421-4_22)
19. Frehse, G.: PHAVer: algorithmic verification of hybrid systems past HyTech. In: *Proceedings of the 8th Int. Workshop on Hybrid Systems: Computation and Control*, pp. 258–273. Springer (2005). [https://doi.org/10.1007/978-3-540-31954-2\\_17](https://doi.org/10.1007/978-3-540-31954-2_17)
20. Galassi, M., Davies, J., Theiler, J., Gough, B., Jungman, G.: *GNU Scientific Library - Reference Manual, Third Edition, for GSL Version 1.12*. Network Theory Ltd (2009)
21. Hahn, E., Hartmanns, A., Hermanns, H., Katoen, J.P.: A compositional modelling and analysis framework for stochastic hybrid systems. *Formal Methods Syst. Des.* **43** (2013). <https://doi.org/10.1007/s10703-012-0167-z>
22. Hartmanns, A., Hermanns, H.: The modest toolset: an integrated environment for quantitative modelling and verification. In: *Proceedings of the 20th Int. Conf. on Tools and Algorithms for the Construction and Analysis of Systems*, pp. 593–598. Springer (2014). [https://doi.org/10.1007/978-3-642-54862-8\\_51](https://doi.org/10.1007/978-3-642-54862-8_51)
23. Henzinger, T.A.: The theory of hybrid automata. In: *Proceedings of the 11th Annual IEEE Symposium on Logic in Computer Science*, pp. 278–292. IEEE Computer Society (1996). <https://doi.org/10.1109/LICS.1996.561342>
24. Henzinger, T.A., Kopke, P.W., Puri, A., Varaiya, P.: What's decidable about hybrid automata? *J. Comput. Syst. Sci.* **57**(1), 94–124 (1998). <https://doi.org/10.1006/jcss.1998.1581>
25. Hermanns, H., Krčál, J., Nies, G.: How is your satellite doing? Battery kinetics with recharging and uncertainty. *Leibniz Trans. Embed. Syst.* **4**(1), 1–28 (2017). <https://doi.org/10.4230/LITES-v004-i001-a004>
26. Hoogsteen, G., Molderink, A., Smit, G., Hurink, J., Kootstra, B., Schuring, F.: Charging electric vehicles, baking pizzas, and melting a fuse in Lochem. *Int. Conf. Exhib. Electr. Distrib.* **2017**(1), 1629–1633 (2017). <https://doi.org/10.1049/oap-cired.2017.0340>
27. Jongerden, M., Haverkort, B., Bohnenkamp, H., Katoen, J.P.: Maximizing system lifetime by battery scheduling. In: *Proceedings of the 2009 IEEE/IFIP Int. Conf. on Dependable Systems & Networks*, pp. 63–72 (2009). <https://doi.org/10.1109/DSN.2009.5270351>
28. Jongerden, M., Mereacre, A., Bohnenkamp, H., Haverkort, B., Katoen, J.P.: Computing optimal schedules of battery usage in embedded systems. *IEEE Trans. Industr. Inf.* **6**(3), 276–286 (2010). <https://doi.org/10.1109/TII.2010.2051813>
29. Jongerden, M.: *Model-based energy analysis of battery powered systems*. Phd thesis - research UT, graduation UT, University of Twente (2010). <https://doi.org/10.3990/1.9789036531146>
30. Jongerden, M., Haverkort, B.: Which battery model to use? *IET Softw.* **3**(6), 445–457 (2009). <https://doi.org/10.1049/iet-sen.2009.0001>
31. Klenke, A.: *Probability Theory: A Comprehensive Course*. Springer, Cham (2014). [https://doi.org/10.1007/978-1-4471-5361-0\\_1](https://doi.org/10.1007/978-1-4471-5361-0_1)
32. Lahiri, K., Raghunathan, A., Dey, S., Panigrahi, D.: Battery-driven system design: a new frontier in low power design. In: *Proceedings the 7th Asia and South Pacific Design Automation Conf. and 15h Int. Conf. on VLSI Design*, pp. 261–267 (2002). <https://doi.org/10.1109/ASPAC.2002.994932>

33. Legay, A., Sedwards, S., Traonouez, L.-M.: Scalable verification of Markov decision processes. In: Canal, C., Idani, A. (eds.) SEFM 2014. LNCS, vol. 8938, pp. 350–362. Springer, Cham (2015). [https://doi.org/10.1007/978-3-319-15201-1\\_23](https://doi.org/10.1007/978-3-319-15201-1_23)
34. Lygeros, J., Prandini, M.: Stochastic hybrid systems: a powerful framework for complex, large scale applications. *Eur. J. Control* **16**(6), 583–594 (2010). <https://doi.org/10.3166/ejc.16.583-594>
35. Manwell, J.F., McGowan, J.G.: Lead acid battery storage model for hybrid energy systems. *Sol. Energy* **50**(5), 399–405 (1993). [https://doi.org/10.1016/0038-092X\(93\)90060-2](https://doi.org/10.1016/0038-092X(93)90060-2)
36. MathWorks: MATLAB Simulink. [www.mathworks.com/products/simulink.html](http://www.mathworks.com/products/simulink.html)
37. Milani, S., Topin, N., Veloso, M., Fang, F.: Explainable reinforcement learning: a survey and comparative review. *ACM Comput. Surv.* **56**(7) (2024). <https://doi.org/10.1145/3616864>
38. Mnih, V., et al.: Human-level control through deep reinforcement learning. *Nature* **518**(7540), 529–533 (2015). <https://doi.org/10.1038/nature14236>
39. Niehage, M., Hartmanns, A., Remke, A.: Learning optimal decisions for stochastic hybrid systems. In: *Proceedings of the 19th ACM-IEEE Int. Conf. on Formal Methods and Models for System Design*, p. 44–55. Association for Computing Machinery (2021). <https://doi.org/10.1145/3487212.3487339>
40. Niehage, M., Remke, A.: Learning that grid-convenience does not hurt resilience in the presence of uncertainty. In: *Proceedings of the 20th Int. Conf. on Formal Modeling and Analysis of Timed Systems*, pp. 298–306. Springer (2022). [https://doi.org/10.1007/978-3-031-15839-1\\_17](https://doi.org/10.1007/978-3-031-15839-1_17)
41. Nies, G., et al.: Mastering operational limitations of LEO satellites – the GOMX-3 approach. *Acta Astronautica* **151**, 726–735 (2018). <https://doi.org/10.1016/j.actaastro.2018.04.040>
42. Rakhmatov, D., Vruthula, S.: An analytical high-level battery model for use in energy management of portable electronic systems. In: *Proceedings of the 2001 Int. Conf. on Computer Aided Design*, pp. 488–493 (2001). <https://doi.org/10.1109/ICCAD.2001.968687>
43. Schneider, S., Wagner, D.H.: Error detection in redundant systems. In: *Proceedings of the Western Joint Computer Conf*, pp. 115–121. Association for Computing Machinery (1957). <https://doi.org/10.1145/1455567.1455587>
44. Schupp, S., Ábrahám, E., Makhoulouf, I.B., Kowalewski, S.: HyPro: a C++ library of state set representations for hybrid systems reachability analysis. In: *Proceedings of the 9th Int. Symposium on NASA Formal Methods*, pp. 288–294. Springer (2017). [https://doi.org/10.1007/978-3-319-57288-8\\_20](https://doi.org/10.1007/978-3-319-57288-8_20)
45. Shmarov, F., Zuliani, P.: ProbReach: verified probabilistic  $\delta$ -reachability for stochastic hybrid systems. In: *Proceedings of the 18th Int. Conf. on Hybrid Systems: Computation and Control*, pp. 134–139. Association for Computing Machinery (2015). <https://doi.org/10.1145/2728606.2728625>
46. da Silva, C., Schupp, S., Remke, A.: Optimizing reachability probabilities for a restricted class of stochastic hybrid automata via Flowpipe-construction. *ACM Trans. Model. Comput. Simul.* **33**(4) (2023). <https://doi.org/10.1145/3607197>
47. Soudjani, S.E.Z., Gevaerts, C., Abate, A.: FAUST<sup>2</sup> : formal abstractions of uncountable-state stochastic processes. In: *Proceedings of the 21st Int. Conf. on Tools and Algorithms for the Construction and Analysis of Systems*, pp. 272–286. Springer (2015). [https://doi.org/10.1007/978-3-662-46681-0\\_23](https://doi.org/10.1007/978-3-662-46681-0_23)
48. Stock, G., Fraire, J.A., Mömke, T., Hermanns, H., Babayev, F., Cruz, E.: Managing fleets of LEO satellites: nonlinear, optimal, efficient, scalable, usable, and robust.

- IEEE Trans. Comput. Aided Des. Integr. Circuits Syst. **39**(11), 3762–3773 (2020). <https://doi.org/10.1109/TCAD.2020.3012751>
49. Sutton, R.S., Barto, A.G.: Reinforcement Learning: An Introduction, 2nd edn. The MIT Press (2018)
  50. The MathWorks: Reinforcement Learning Toolbox. <https://www.mathworks.com/products/reinforcement-learning.html>
  51. Watkins, C., Dayan, P.: Q-learning. Mach. Learn. **8**(3), 279–292 (1992). <https://doi.org/10.1007/BF00992698>
  52. Willems, J.C.: From time series to linear system – part i. finite dimensional linear time invariant systems. Automatica **22**(5), 561–580 (1986). [https://doi.org/10.1016/0005-1098\(86\)90066-X](https://doi.org/10.1016/0005-1098(86)90066-X)
  53. Willemsen, L., Remke, A., Ábrahám, E.: Comparing two approaches to include stochasticity in hybrid automata. In: Proceedings of the 20th Int. Conf. on Quantitative Evaluation of Systems, pp. 238–254. Springer Nature (2023). [https://doi.org/10.1007/978-3-031-43835-6\\_17](https://doi.org/10.1007/978-3-031-43835-6_17)
  54. Willemsen, L., Remke, A., Ábrahám, E.: (de-)composed and more: Eager and lazy specifications (camels) for stochastic hybrid systems. In: Principles of Verification: Cycling the Probabilistic Landscape : Essays Dedicated to Joost-Pieter Katoen on the Occasion of His 60th Birthday, Part III. Springer Nature (2025). [https://doi.org/10.1007/978-3-031-75778-5\\_15](https://doi.org/10.1007/978-3-031-75778-5_15)
  55. Wilson, E.B.: Probable inference, the law of succession, and statistical inference. J. Am. Stat. Assoc. **22**(158), 209–212 (1927). <https://doi.org/10.1080/01621459.1927.10502953>
  56. Wognsen, E.R., Haverkort, B.R., Jongerden, M., Hansen, R.R., Larsen, K.G.: A score function for optimizing the cycle-life of battery-powered embedded systems. In: Sankaranarayanan, S., Vicario, E. (eds.) FORMATS 2015. LNCS, vol. 9268, pp. 305–320. Springer, Cham (2015). [https://doi.org/10.1007/978-3-319-22975-1\\_20](https://doi.org/10.1007/978-3-319-22975-1_20)